

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой

ИУ5

(индекс)

В.И. Терехов

(И.О. Фамилия)

(подпись)

(дата)

ЗАДАНИЕ
на выполнение научно-исследовательской работы

по теме Разработка и оценка моделей машинного обучения

Студент группы ИУ5-63Б

Иванов Николай Сергеевич

Направленность НИР (учебная, исследовательская, практическая, производственная, др.)

ИССЛЕДОВАТЕЛЬСКАЯ

Источник тематики (кафедра, предприятие, НИР) КАФЕДРА

График выполнения НИР:

25% к _____ нед., 50% к _____ нед., 75% к _____ нед., 75% к _____ нед

Техническое задание: решение задачи машинного обучения на основе материалов

дисциплины. Выбор датасета, первичный анализ, выбор метрик для оценки качества моделей, построение базового решения, оценка качества, подбор гиперпараметров.

Оформление научно-исследовательской работы: _____

Расчетно-пояснительная записка на _____ листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)

Дата выдачи задания «07» февраля 2025 г.

Руководитель НИР

(подпись, дата)

Ю.Е. Гапанюк

(И.О. Фамилия)

Студент

(подпись, дата)

Н.С. Иванов

(И.О. Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. ПОСТАНОВКА ЗАДАЧИ	6
2. АНАЛИЗ ДАТАСЕТА	8
3. ВЫБОР МОДЕЛЕЙ И МЕТРИК ДЛЯ ОЦЕНКИ КАЧЕСТВА.....	12
4. ПОСТРОЕНИЕ БАЗОВОГО РЕШЕНИЯ.....	14
5. ПОДБОР ГИПЕРПАРАМЕТРОВ	15
6. ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ	17
7. ВЕБ-ПРИЛОЖЕНИЕ	20
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	27

ВВЕДЕНИЕ

В современной игровой индустрии анализ факторов, влияющих на коммерческий успех видеоигр, становится все более важным для разработчиков и издателей, стремящихся максимизировать свою прибыль в условиях высококонкурентного рынка. Понимание ключевых характеристик, определяющих выручку игр на платформе Steam, является критически важным для принятия стратегических решений в области разработки, ценообразования и маркетинга. В рамках представленной работы мной было проведено исследование, целью которого был анализ различных моделей машинного обучения, которые могут быть использованы для прогнозирования потенциальной выручки игр на основе их характеристик.

Исследование было сосредоточено на анализе набора данных `Steam_2024_bestRevenue_1500`, содержащего информацию о 1500 играх с наибольшей выручкой на платформе Steam. Датасет включает такие характеристики, как название игры, дата выпуска, количество проданных копий, цена, выручка, среднее время игры, оценка пользователей, класс издателя, а также информацию об издателях и разработчиках. Целью работы стало построение и оценка моделей машинного обучения для прогнозирования выручки игр на основе имеющихся данных.

Модели машинного обучения представляют собой мощный инструмент для анализа сложных взаимосвязей между различными характеристиками игр и их влиянием на коммерческий успех. Представленное исследование включает в себя всесторонний анализ данных, включая предварительное исследовательское изучение, выбор и обработку признаков, корреляционный анализ, а также построение и сравнение различных моделей машинного обучения с подбором оптимальных гиперпараметров.

Результаты работы могут быть использованы разработчиками и издателями игр для оценки потенциальной выручки новых проектов, оптимизации стратегий ценообразования и маркетинга, а также для более глубокого понимания факторов, влияющих на коммерческий успех игр на платформе Steam. Кроме того, методология и выводы исследования могут быть применены для анализа данных и в других сегментах цифровой дистрибуции контента.

1. ПОСТАНОВКА ЗАДАЧИ

Целью научно-исследовательской работы является разработка и оценка моделей машинного обучения для прогнозирования выручки игр. В качестве целевой переменной в задаче регрессии использована переменная 'revenue', отражающая фактическую выручку игр на платформе Steam. Исследование направлено на выявление ключевых факторов, влияющих на коммерческий успех игр, и создание моделей, способных с высокой точностью предсказывать потенциальную выручку на основе различных характеристик игровых продуктов.

Датасет включает в себя следующие признаки:

Поле	Описание	Тип данных
name	Название игры	string
releaseDate	Дата выпуска игры	datetime
copiesSold	Количество проданных копий	float64
price	Цена игры	float64
revenue	Выручка от продаж	float64
avgPlaytime	Среднее время игры	float64
reviewScore	Оценка пользователей	float64
publisherClass	Класс издателя (Indie, AA, AAA)	string
publishers	Названия издателей	string
developers	Названия разработчиков	string

Поле	Описание		Тип данных
steamId	Идентификатор игры в Steam		string
Признаки, созданные в процессе анализа			
releaseYear	Год выпуска игры	int32	
releaseMonth	Месяц выпуска игры	int32	
revenuePerCopy	Выручка на копию	float64	

2. АНАЛИЗ ДАТАСЕТА

Обработка пропусков:

```
# Проверка пропусков
print("Пропущенные значения:")
missing_data = df.isnull().sum()
print(missing_data[missing_data > 0])
```

Рисунок 1 – Результаты анализа пропусков в данных.

```
# Заполнение пропусков
df_processed = df.copy()

# Для числовых переменных - медиана
numeric_cols = ['copiesSold', 'price', 'revenue', 'avgPlaytime', 'reviewScore']
for col in numeric_cols:
    if df_processed[col].isnull().sum() > 0:
        median_val = df_processed[col].median()
        df_processed[col].fillna(median_val, inplace=True)
        print(f"Заполнили пропуски в {col} медианой: {median_val}")

# Для категориальных переменных - мода
categorical_cols = ['publisherClass', 'publishers', 'developers']
for col in categorical_cols:
    if df_processed[col].isnull().sum() > 0:
        mode_val = df_processed[col].mode()[0]
        df_processed[col].fillna(mode_val, inplace=True)
        print(f"Заполнили пропуски в {col} модой: {mode_val}")

print(f"\nПропуски после обработки:")
print(df_processed.isnull().sum().sum())
```

Рисунок 2 – Заполнение пропусков

3. АНАЛИЗ И ОБРАБОТКА ПРОПУСКОВ

Пропуски после обработки:
0

Рисунок 3 – Подсчет пропусков после обработки

Построим **диаграммы типа** для иллюстрации распределения значений признаков.

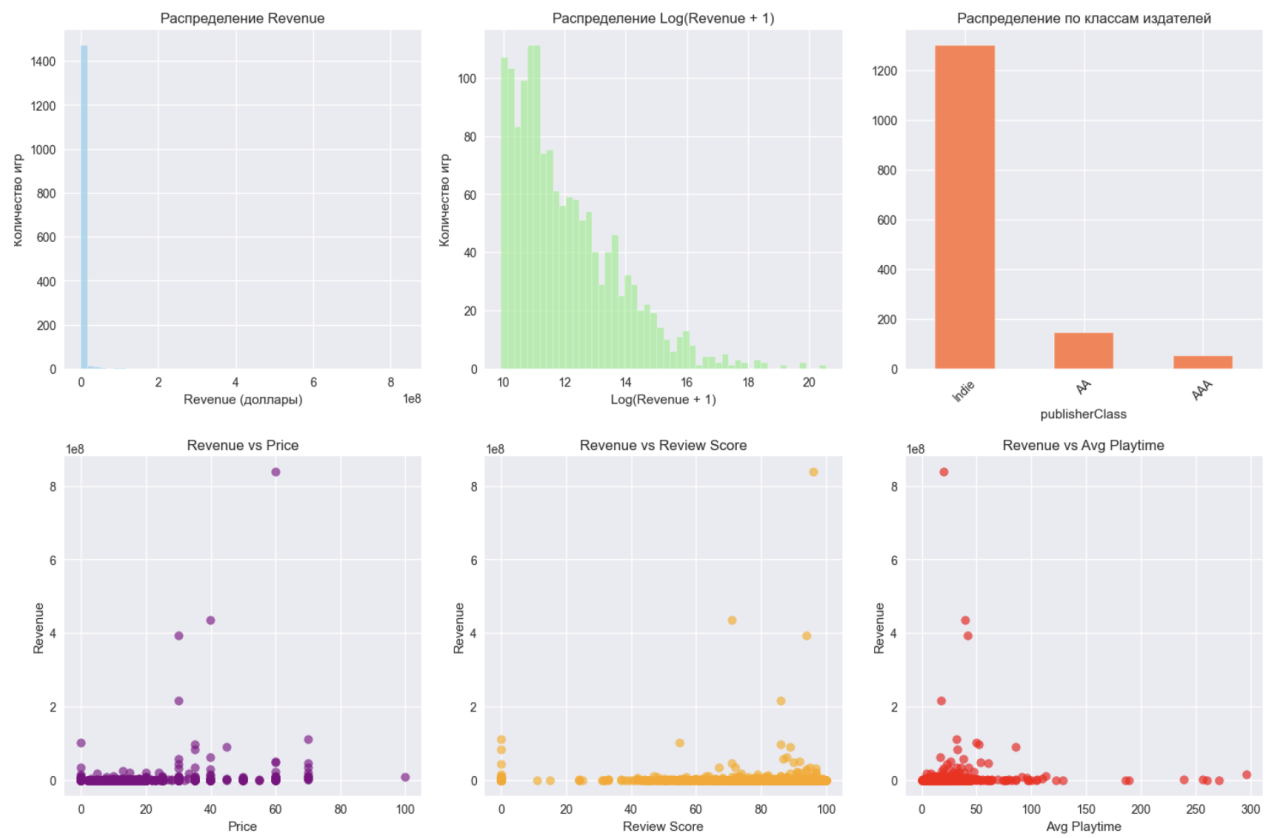


Рисунок 4 – Распределение значений признаков.

Построим корреляционную матрицу признаков:

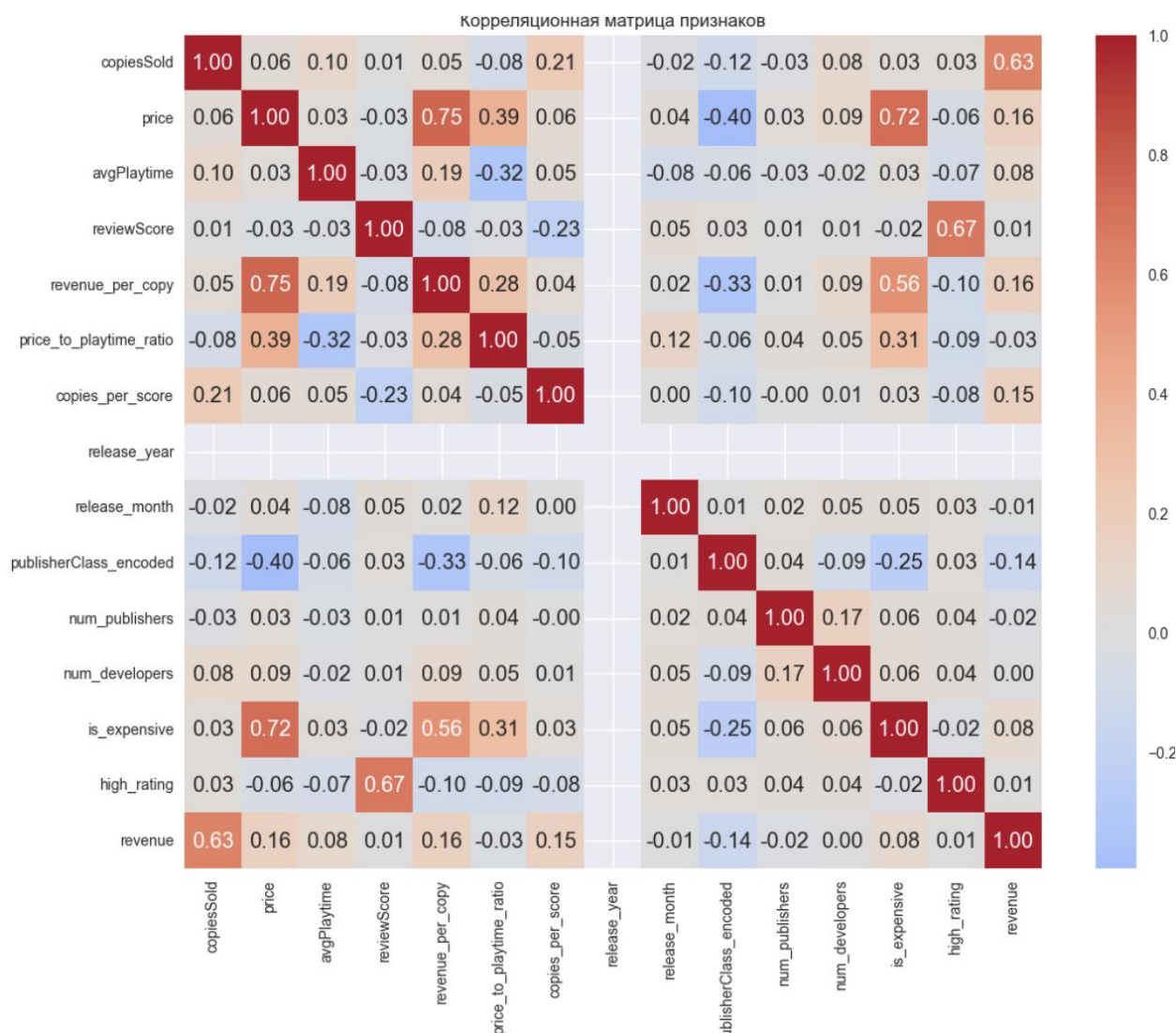


Рисунок 5 – Корреляционная матрица признаков.

Анализ корреляционной матрицы показывает, что только некоторые признаки имеют значимую корреляцию с целевым признаком 'revenue'. Наиболее сильная корреляция наблюдается между выручкой и количеством проданных копий ('copiesSold', 0.63), что логично, поскольку количество продаж напрямую влияет на общую выручку. Остальные признаки демонстрируют слабую или умеренную корреляцию с целевой переменной.

При этом в датасете присутствуют группы признаков с сильной корреляцией между собой, что указывает на наличие избыточной информации. Например, наблюдается сильная корреляция между 'price' и 'is_expensive' (0.72), между 'reviewScore' и 'high_rating' (0.67), а также между

'revenue_per_copy' и 'price' (0.75). Такие высокие корреляции могут привести к мультиколлинеарности при построении моделей, что потенциально может негативно повлиять на их качество и интерпретируемость.

3. ВЫБОР МОДЕЛЕЙ И МЕТРИК ДЛЯ ОЦЕНКИ КАЧЕСТВА

Для оценки качества моделей выберем следующие метрики:

Метрики регрессии:

- **RMSE (Root Mean Square Error)** - среднеквадратичная ошибка.

Показывает среднюю величину ошибки предсказания выручки в долларах.

Чувствительна к выбросам, что важно для выявления больших ошибок в прогнозировании высокодоходных игр.

- **MAE (Mean Absolute Error)** - средняя абсолютная ошибка.

Показывает среднюю абсолютную разность между предсказанной и фактической выручкой. Менее чувствительна к выбросам по сравнению с RMSE.

- **R^2 (коэффициент детерминации)** - показывает долю дисперсии выручки, объясняемую моделью. Значения ближе к 1 указывают на лучшее качество модели.

Метрики классификации (через преобразование задачи):

Для дополнительной оценки качества моделей преобразуем задачу регрессии в задачу классификации, разделив игры на три категории по уровню выручки: низкая, средняя и высокая выручка.

- **Accuracy (Точность общего предсказания)** - доля правильно предсказанных категорий выручки от общего числа игр. Показывает, насколько точно модель определяет диапазон выручки игры.

- **Precision (Точность)** - доля правильных предсказаний для каждой категории выручки среди всех предсказанных в этой категории. Важна для понимания надежности предсказаний модели для конкретных диапазонов выручки.

- **Weighted F1-score** - среднее значение F1-метрики с учетом частоты каждой категории выручки. Обеспечивает сбалансированную оценку качества модели для всех диапазонов выручки.

- **Recall (Полнота)** - показывает, насколько хорошо модель находит все игры, которые действительно принадлежат к определенной категории выручки. Важна для оценки способности модели не пропускать игры с высокой потенциальной выручкой.

С учетом следующих факторов:

- Требование задания НИРС: не менее 5 моделей, не менее 2 ансамблевых моделей;
- Решение задачи регрессии и классификации (преобразование задачи);
- Датасет содержит числовые признаки.

выберем следующие модели:

- Линейная регрессия;
- Гребневая регрессия;
- Дерево решений;
- Случайный лес;
- Градиентный бустинг;
- AdaBoost;
- Метод опорных векторов.

4. ПОСТРОЕНИЕ БАЗОВОГО РЕШЕНИЯ

Построим базовое решение для каждой из моделей:

```
# 9. ПОСТРОЕНИЕ БАЗОВЫХ МОДЕЛЕЙ (BASELINE)
print("\n\n9. ПОСТРОЕНИЕ БАЗОВЫХ МОДЕЛЕЙ")
print("-" * 50)

# Инициализация моделей
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(random_state=42),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42, n_estimators=100),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
    'AdaBoost': AdaBoostRegressor(random_state=42),
    'SVR': SVR()
}

baseline_results = []

print("Обучение базовых моделей...")
for name, model in models.items():
    print(f"Обучение {name}...")

    if name == 'SVR':
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    result = evaluate_model(y_test, y_pred, name)
    baseline_results.append(result)

baseline_df = pd.DataFrame(baseline_results)
print("\nРезультаты базовых моделей:")
print(baseline_df.round(4))
```

Рисунок 6 – Базовое решение.

```
Обучение Ridge Regression...
Обучение Decision Tree...
Обучение Random Forest...
Обучение Gradient Boosting...
Обучение AdaBoost...
Обучение SVR...
```

Результаты базовых моделей:					
	Model	RMSE	MAE	R ²	MAPE (%) \
0	Linear Regression	1.093019e+07	4.042406e+06	0.7906	3344.4657
1	Ridge Regression	1.092658e+07	4.037061e+06	0.7907	3338.3595
2	Decision Tree	1.255994e+07	1.490749e+06	0.7234	14.5051
3	Random Forest	3.678916e+06	3.899565e+05	0.9763	6.2877
4	Gradient Boosting	2.605975e+06	4.957458e+05	0.9881	71.6009
5	AdaBoost	1.757233e+07	3.597896e+06	0.4587	2741.9372
6	SVR	2.404999e+07	2.887369e+06	-0.0140	122.9079

	CV Residuals (%)	Accuracy	Precision	Recall	F1-score
0	372.1081	0.5067	0.5036	0.5067	0.4259
1	371.9854	0.5067	0.5036	0.5067	0.4259
2	426.8909	0.9467	0.9468	0.9467	0.9467
3	125.2075	0.9767	0.9771	0.9767	0.9767
4	88.6834	0.7533	0.7881	0.7533	0.7518
5	597.8058	0.3400	0.1156	0.3400	0.1725
6	813.0860	0.3300	0.1089	0.3300	0.1638

Рисунок 7 – Оценка точности базового решения.

5. ПОДБОР ГИПЕРПАРАМЕТРОВ

Для повышения точности моделей произведём подбор гиперпараметров моделей с использованием функции **GridSearchCV** и оптимизацию.

```
# 10. ПОДБОР ГИПЕРПАРАМЕТРОВ
print("\n\n10. ПОДБОР ГИПЕРПАРАМЕТРОВ")
print("-" * 50)

# Параметры для Grid Search
param_grids = {
    'Random Forest': {
        'n_estimators': [50, 100, 200],
        'max_depth': [5, 10, None],
        'min_samples_split': [2, 5, 10]
    },
    'Gradient Boosting': {
        'n_estimators': [50, 100, 200],
        'learning_rate': [0.01, 0.1, 0.2],
        'max_depth': [3, 5, 7]
    },
    'Ridge Regression': {
        'alpha': [0.1, 1, 10, 100]
    }
}

optimized_models = {}
optimized_results = []

for name in ['Random Forest', 'Gradient Boosting', 'Ridge Regression']:
    print(f"Оптимизация {name}...")

    model = models[name]
    param_grid = param_grids[name]

    grid_search = GridSearchCV(
        model, param_grid,
        cv=5, scoring='neg_mean_squared_error',
        n_jobs=-1, verbose=0
    )

    grid_search.fit(X_train, y_train)

    best_model = grid_search.best_estimator_
    optimized_models[name] = best_model

    y_pred = best_model.predict(X_test)
    result = evaluate_model(y_test, y_pred, f"{name} (Optimized)")
    optimized_results.append(result)

    print(f"Лучшие параметры для {name}: {grid_search.best_params_}")

# Добавляем неоптимизированные модели
for name in ['Linear Regression', 'Decision Tree', 'AdaBoost']:
    model = models[name]
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    result = evaluate_model(y_test, y_pred, f"{name} (Baseline)")
    optimized_results.append(result)
```

```
result = evaluate_model(y_test, y_pred, f"{name} (Baseline)")
optimized_results.append(result)

# SVR с оптимизацией
print("Оптимизация SVR...")
svr_params = {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto'], 'kernel': ['rbf', 'linear']}
svr_grid = GridSearchCV(SVR(), svr_params, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
svr_grid.fit(X_train_scaled, y_train)
svr_pred = svr_grid.predict(X_test_scaled)
svr_result = evaluate_model(y_test, svr_pred, "SVR (Optimized)")
optimized_results.append(svr_result)

optimized_df = pd.DataFrame(optimized_results)
print("\nРезультаты после оптимизации:")
print(optimized_df.round(4))
```

```
Оптимизация Random Forest...
Лучшие параметры для Random Forest: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 50}
Оптимизация Gradient Boosting...
Лучшие параметры для Gradient Boosting: {'learning_rate': 0.2, 'max_depth': 3, 'n_estimators': 200}
Оптимизация Ridge Regression...
Лучшие параметры для Ridge Regression: {'alpha': 100}
Оптимизация SVR...
```

Рисунок 8 – Подбор гиперпараметров и оптимизация.

6. ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ

Построим диаграммы, отображающие оценки качества исследуемых моделей до и после подбора гиперпараметров:

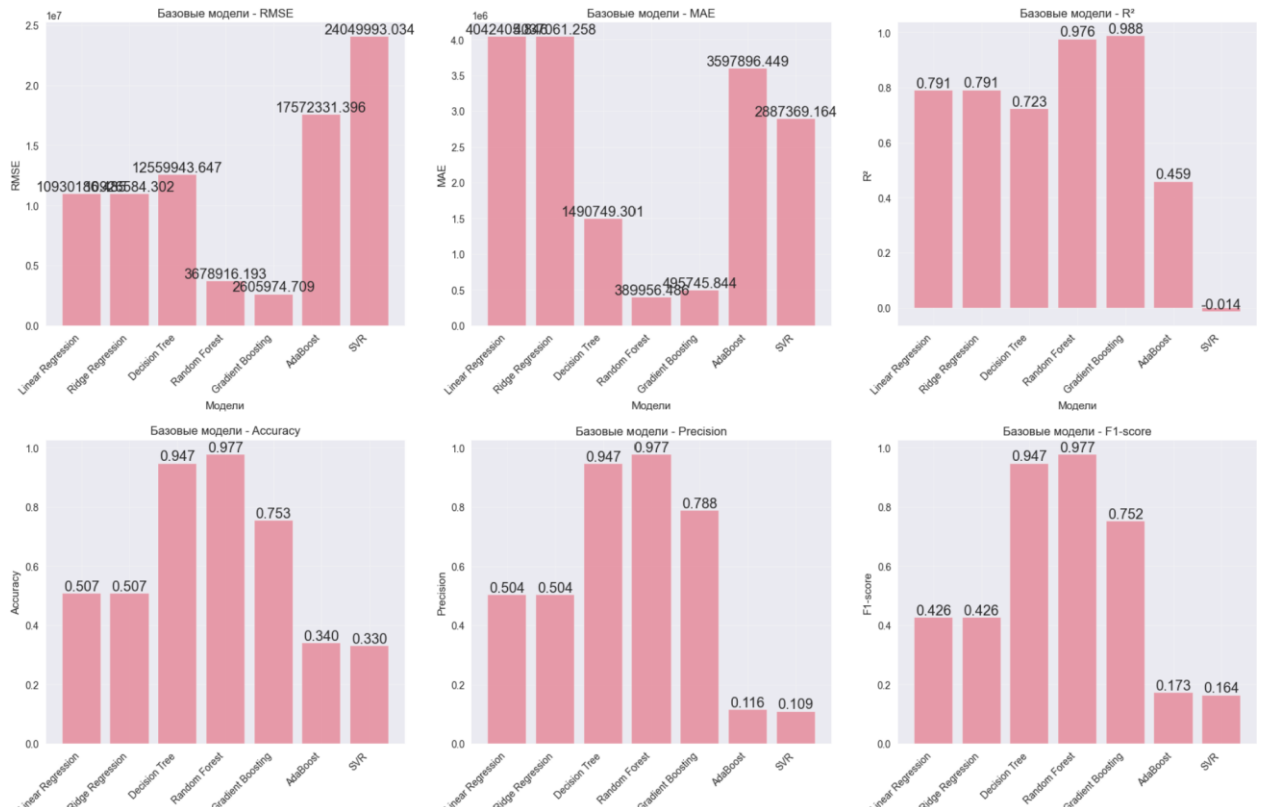


Рисунок 9 – Оценка моделей до оптимизации.

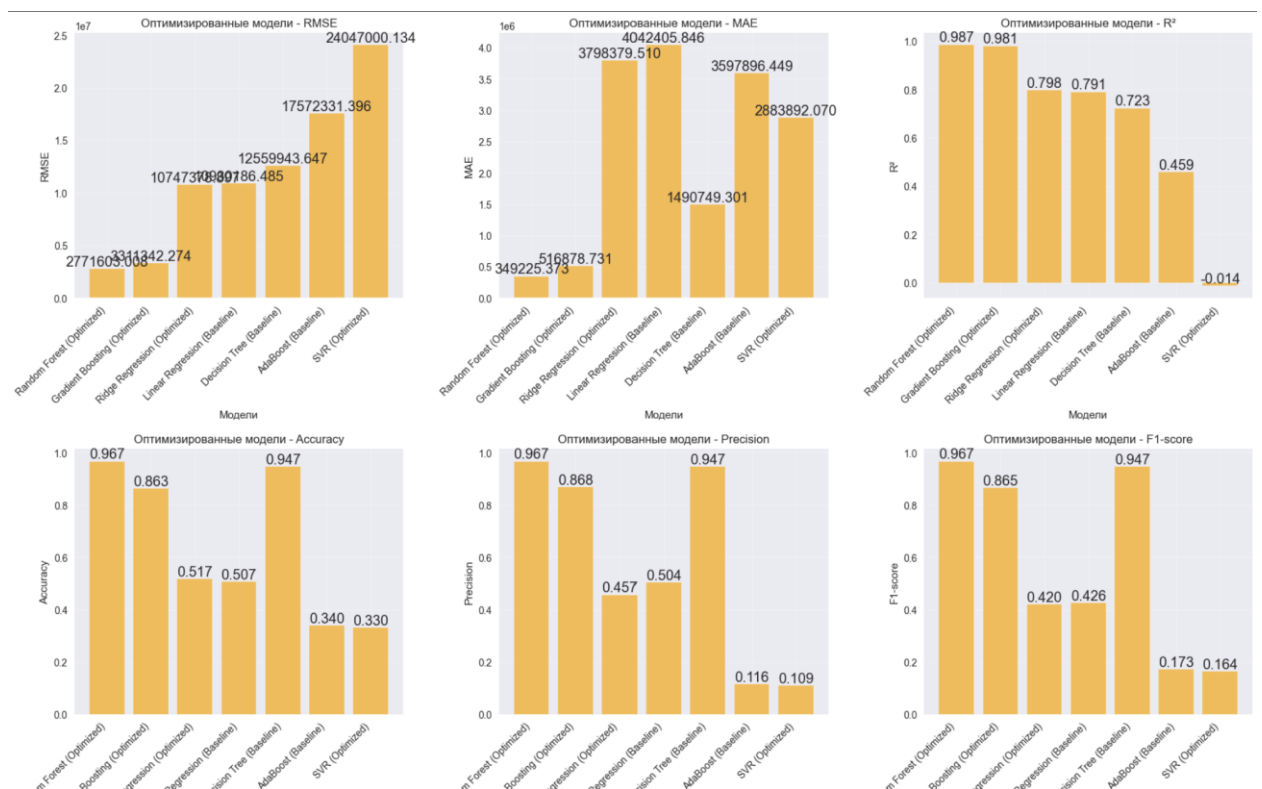


Рисунок 10 – Оценка моделей после оптимизации.

В ходе исследования наибольшую точность (**0.967**) по всем анализируемым метрикам показала модель **Случайный лес (Random Forest)**.

7. ДОПОЛНИТЕЛЬНЫЕ ИЛЛЮСТРАЦИИ

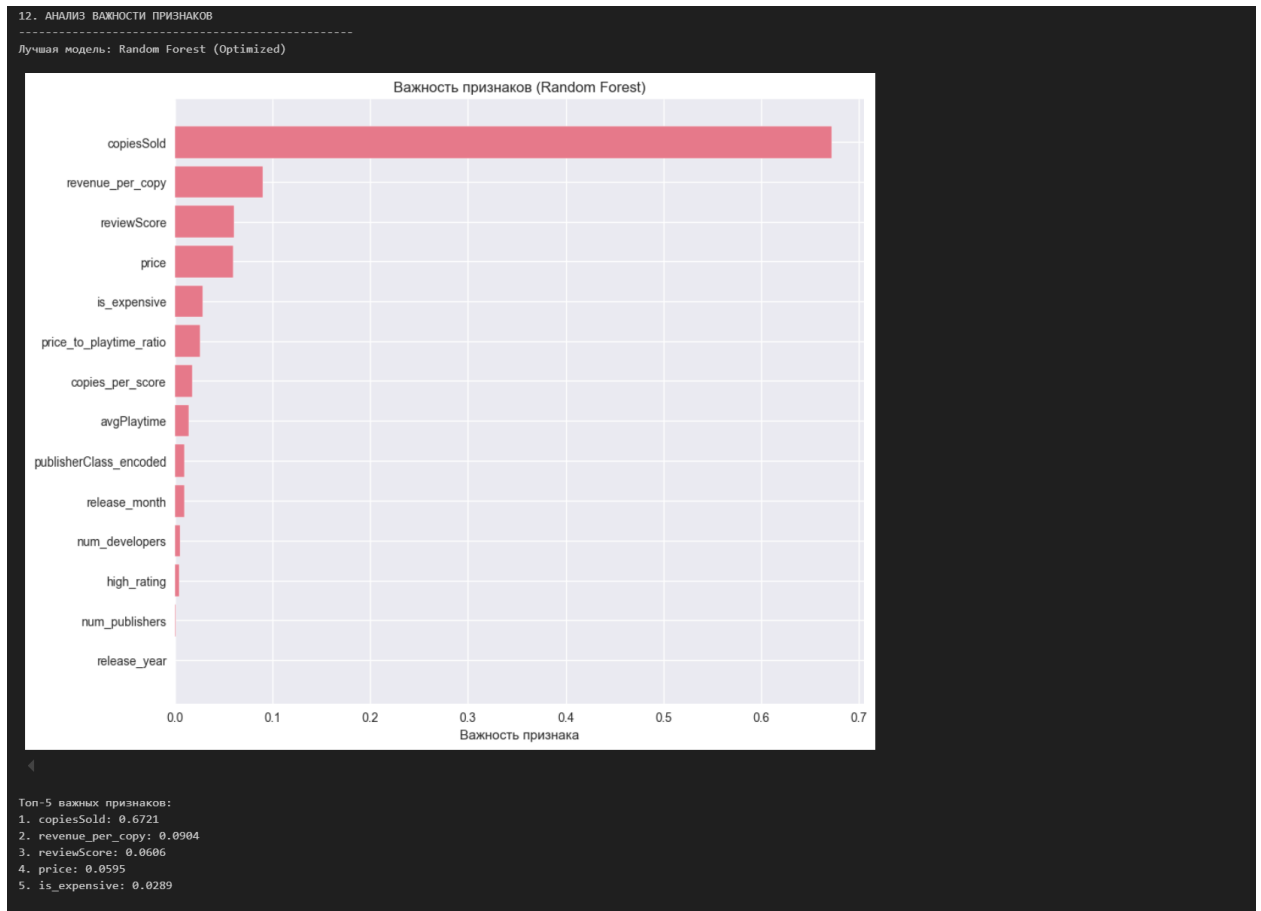


Рисунок 11 – Важность признаков.

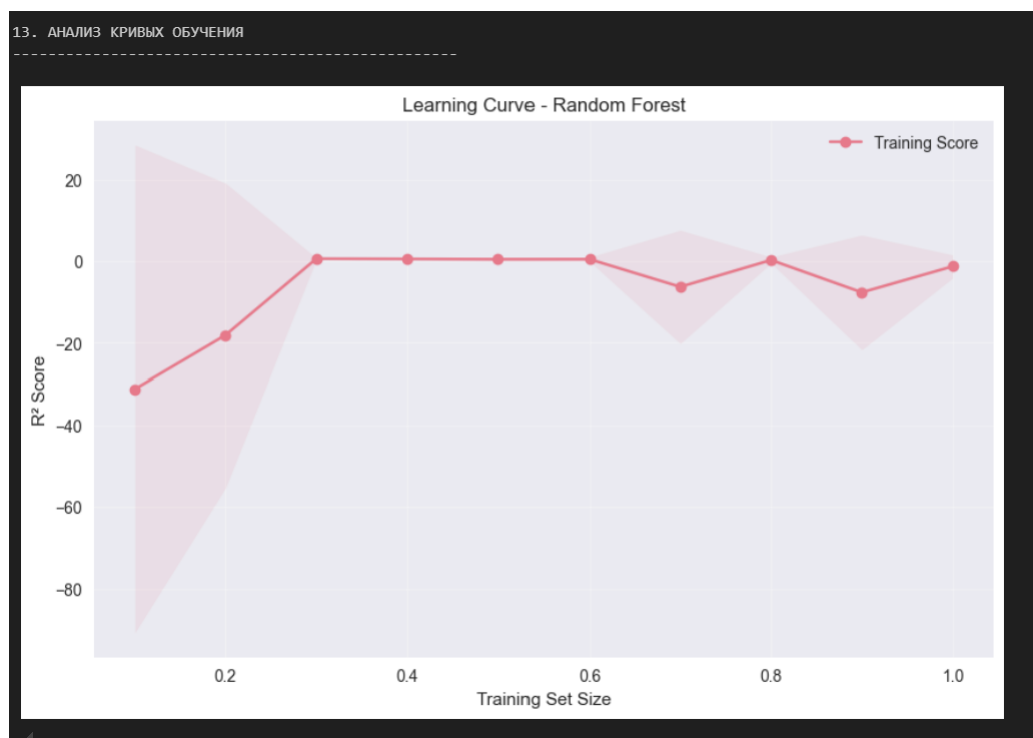


Рисунок 12 – Кривая обучения.

8. ВЕБ-ПРИЛОЖЕНИЕ

Реализуем веб-приложение для демонстрации влияния гиперпараметров на точность модели **Линейной регрессии, Случайный лес, Дерево решений**. Используем фреймворк **Streamlit**.

```
import streamlit as st
st.set_page_config(layout="wide", page_title="ML Model Comparison for Game Revenue Prediction")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.inspection import partial_dependence

# Загрузка данных
@st.cache_data
def load_data():
    data = pd.read_csv('Steam_2024_bestRevenue_1500 (3).csv')
    data['releaseDate'] = pd.to_datetime(data['releaseDate'], format='%d-%m-%Y')
    data['logRevenue'] = np.log1p(data['revenue']) # Логарифмирование для нормализации
    data['logCopiesSold'] = np.log1p(data['copiesSold'])
    return data

data = load_data()

# Настройка страницы
# st.set_page_config(layout="wide", page_title="ML Model Comparison for Game Revenue Prediction")
st.title('ML Сравнение моделей машинного обучения для прогнозирования выручки видеоигр')
st.write("""
Это приложение позволит сравнивать разные модели машинного обучения для предсказания выручки видеоигр.
Вы можете выбирать модели, настраивать их гиперпараметры и анализировать результаты.
""")

# Разделение на колонки
col1, col2 = st.columns([1, 3])

# Настройка модели в сайдбаре
with st.sidebar:
    st.header('Настройки модели')

    # Выбор признаков
    feature_options = ['copiesSold', 'price', 'avgPlaytime', 'reviewScore', 'logCopiesSold']
    feature = st.selectbox('Выберите признак:', feature_options)

    # Выбор целевой переменной
    target = st.radio('Целевая переменная:', ['revenue', 'logRevenue'])

    # Выбор моделей для сравнения
    st.subheader('Выбор моделей')
    use_linear = st.checkbox('Линейная регрессия', True)
    use_tree = st.checkbox('Дерево решений', True)
    use_forest = st.checkbox('Случайный лес', True)

    # Гиперпараметры для каждой модели
    st.subheader('Гиперпараметры')
```

```

# Гиперпараметры для каждой модели
st.subheader('Гиперпараметры')

if use_linear:
    st.markdown("***Линейная регрессия***")
    linear_degree = st.slider('Степень полинома:', 1, 5, 1, key='linear_degree')
    linear_alpha = st.slider('Alpha (регуляризация):', 0.0, 10.0, 0.0, 0.1, key='linear_alpha')

if use_tree:
    st.markdown("***Дерево решений***")
    tree_depth = st.slider('Глубина дерева:', 1, 20, 5, key='tree_depth')
    tree_min_samples = st.slider('Минимальное число образцов в листе:', 1, 20, 2, key='tree_min_samples')

if use_forest:
    st.markdown("***Случайный лес***")
    forest_n_estimators = st.slider('Число деревьев:', 1, 200, 100, key='forest_n_estimators')
    forest_max_depth = st.slider('Макс. глубина:', 1, 50, 10, key='forest_max_depth')

# Разделение данных
X = data[[feature]].values
y = data[target].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Создание и обучение моделей
models = {}
results = []

if use_linear:
    if linear_alpha > 0:
        model = make_pipeline(
            PolynomialFeatures(degree=linear_degree),
            StandardScaler(),
            Ridge(alpha=linear_alpha)
        )
        model_name = f"Ridge Regression (degree={linear_degree},  $\alpha$ ={linear_alpha})"
    else:
        model = make_pipeline(
            PolynomialFeatures(degree=linear_degree),
            LinearRegression()
        )
        model_name = f"Linear Regression (degree={linear_degree})"

    model.fit(X_train, y_train)
    models[model_name] = model
    y_pred = model.predict(X_test)
    results.append({
        'Model': model_name,
        'R²': r2_score(y_test, y_pred),
        'MSE': mean_squared_error(y_test, y_pred),
        'MAE': mean_absolute_error(y_test, y_pred)
    })

if use_tree:
    model = DecisionTreeRegressor(
        max_depth=tree_depth,
        min_samples_leaf=tree_min_samples,
        random_state=42
    )
    model_name = f"Decision Tree (depth={tree_depth}, min_samples={tree_min_samples})"
    model.fit(X_train, y_train)
    models[model_name] = model

```

```

models[model_name] = model
y_pred = model.predict(X_test)
results.append({
    'Model': model_name,
    'R²': r2_score(y_test, y_pred),
    'MSE': mean_squared_error(y_test, y_pred),
    'MAE': mean_absolute_error(y_test, y_pred)
})

if use_forest:
    model = RandomForestRegressor(
        n_estimators=forest_n_estimators,
        max_depth=forest_max_depth,
        random_state=42
    )
    model_name = f"Random Forest (trees={forest_n_estimators}, depth={forest_max_depth})"
    model.fit(X_train, y_train)
    models[model_name] = model
    y_pred = model.predict(X_test)
    results.append({
        'Model': model_name,
        'R²': r2_score(y_test, y_pred),
        'MSE': mean_squared_error(y_test, y_pred),
        'MAE': mean_absolute_error(y_test, y_pred)
    })

# Отображение результатов
if not models:
    st.warning("Пожалуйста, выберите хотя бы одну модель для сравнения.")
else:
    # Таблица с результатами
    results_df = pd.DataFrame(results).sort_values('R²', ascending=False)
    st.subheader("Сравнение производительности моделей")
    st.dataframe(results_df.style.highlight_max(subset=['R²'], color='lightgreen')
                .highlight_min(subset=['MSE', 'MAE'], color='lightgreen')
                .format({'R²': '{:.3f}', 'MSE': '{:,.0f}', 'MAE': '{:,.0f}'}),
                use_container_width=True)

    # Визуализация предсказаний
    st.subheader("Визуализация предсказаний")

    fig, ax = plt.subplots(figsize=(12, 6))

    # Сортируем для красивого графика
    sorted_idx = np.argsort(X_test[:, 0])
    X_sorted = X_test[sorted_idx]
    y_test_sorted = y_test[sorted_idx]

    # Фактические значения
    ax.scatter(X_sorted, y_test_sorted, color='black', label='Фактические значения', alpha=0.4)

    # Предсказания каждой модели
    colors = plt.cm.rainbow(np.linspace(0, 1, len(models)))
    for (name, model), color in zip(models.items(), colors):
        y_pred_sorted = model.predict(X_sorted)
        ax.plot(X_sorted, y_pred_sorted, label=name, color=color, linewidth=2)

    ax.set_xlabel(feature)
    ax.set_ylabel(target)
    ax.set_title('Сравнение предсказаний моделей')
    ax.legend()

```

```

177     fig, ax = plt.subplots(1, 1)
178     st.pyplot(fig)
179
180     # Partial Dependence Plot для случайного леса (если выбран)
181     if use_forest and len(models) > 0:
182         st.subheader('Анализ важности признаков (только для Random Forest)')
183
184         forest_model = next((m for n, m in models.items() if 'Random Forest' in n))
185
186         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
187
188         # Важность признаков
189         if hasattr(forest_model, 'feature_importances_'):
190             importances = forest_model.feature_importances_
191             features = [feature]
192             ax1.barh(features, importances)
193             ax1.set_title('Важность признаков')
194         else:
195             ax1.text(0.5, 0.5, 'Недоступно для выбранной модели',
196                     ha='center', va='center')
197
198         # Partial Dependence Plot
199         try:
200             pdp = partial_dependence(forest_model, X_train, features=[0])
201             ax2.plot(pdp['values'][0], pdp['average'][0])
202             ax2.set_xlabel(feature)
203             ax2.set_ylabel('Частичная зависимость')
204             ax2.set_title('Partial Dependence Plot')
205             ax2.grid(True)
206         except Exception as e:
207             ax2.text(0.5, 0.5, f'Ошибка: {str(e)}',
208                     ha='center', va='center')
209
210         st.pyplot(fig)
211
212     # Дополнительный анализ данных
213     st.subheader('Дополнительный анализ данных')
214
215     tab1, tab2, tab3 = st.tabs(["Распределение признаков", "Корреляция", "Статистика"])
216
217     with tab1:
218         fig, ax = plt.subplots(figsize=(10, 4))
219         sns.histplot(data[feature], kde=True, ax=ax)
220         ax.set_title(f'Распределение признака {feature}')
221         st.pyplot(fig)
222
223         if target != 'logRevenue':
224             fig, ax = plt.subplots(figsize=(10, 4))
225             sns.histplot(data[target], kde=True, ax=ax)
226             ax.set_title(f'Распределение целевой переменной {target}')
227             st.pyplot(fig)
228
229     with tab2:
230         numeric_cols = data.select_dtypes(include=[np.number]).columns
231         corr = data[numeric_cols].corr()
232
233         fig, ax = plt.subplots(figsize=(10, 8))
234         sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', ax=ax)
235         ax.set_title('Матрица корреляций')
236         st.pyplot(fig)
237

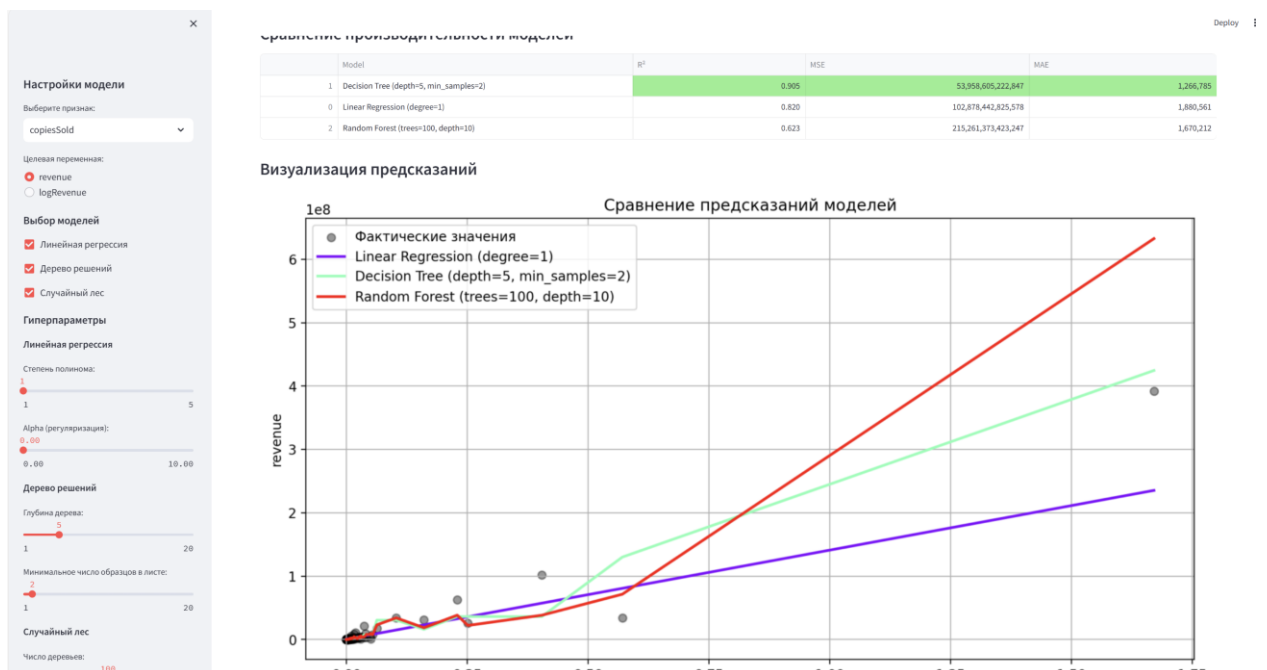
```

```

203         ax2.set_ylabel('Частичная зависимость')
204         ax2.set_title('Partial Dependence Plot')
205         ax2.grid(True)
206     except Exception as e:
207         ax2.text(0.5, 0.5, f'Ошибка: {str(e)}',
208                 ha='center', va='center')
209
210     st.pyplot(fig)
211
212     # Дополнительный анализ данных
213     st.subheader('Дополнительный анализ данных')
214
215     tab1, tab2, tab3 = st.tabs(["Распределение признаков", "Корреляция", "Статистика"])
216
217     with tab1:
218         fig, ax = plt.subplots(figsize=(10, 4))
219         sns.histplot(data[feature], kde=True, ax=ax)
220         ax.set_title(f'Распределение признака {feature}')
221         st.pyplot(fig)
222
223         if target != 'logRevenue':
224             fig, ax = plt.subplots(figsize=(10, 4))
225             sns.histplot(data[target], kde=True, ax=ax)
226             ax.set_title(f'Распределение целевой переменной {target}')
227             st.pyplot(fig)
228
229     with tab2:
230         numeric_cols = data.select_dtypes(include=[np.number]).columns
231         corr = data[numeric_cols].corr()
232
233         fig, ax = plt.subplots(figsize=(10, 8))
234         sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', ax=ax)
235         ax.set_title('Матрица корреляций')
236         st.pyplot(fig)
237
238     with tab3:
239         st.write(f"**Описательная статистика для {feature}**")
240         st.write(data[feature].describe().to_frame().T)
241
242         st.write(f"**Описательная статистика для {target}**")
243         st.write(data[target].describe().to_frame().T)
244
245     # Интерпретация результатов
246     st.subheader('Интерпретация результатов')
247     st.write("""
248     - **R2 (Коэффициент детерминации)**: Чем ближе к 1, тем лучше модель объясняет вариативность данных.
249     - **MSE (Среднеквадратичная ошибка)**: Средний квадрат разницы между предсказанными и фактическими значениями.
250     - **MAE (Средняя абсолютная ошибка)**: Средняя абсолютная разница между предсказанными и фактическими значениями.
251     """)

```

Рисунок 13 – Реализация веб-приложения для демонстрации модели.



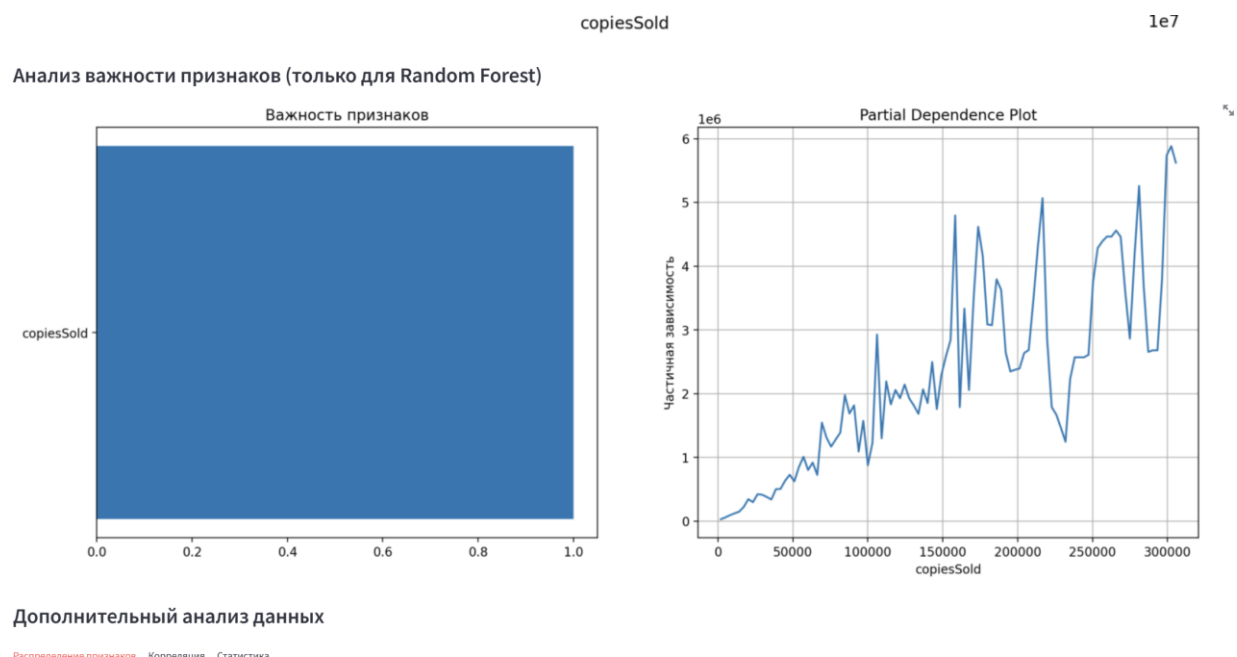


Рисунок 14 – Веб-приложение.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской было проведено исследование задачи определения выручки. Были проанализированы различные модели машинного обучения, в том числе ансамблиевые, а также их оптимизация путём подбора гиперпараметров.

Наиболее высокую точность по исследуемым метрикам показала модель **Случайный лес** после подбора гиперпараметров.

Для демонстрации влияния гиперпараметров на точность модели было разработано веб-приложение.

Таким образом, исследуемые модели машинного обучения и проведенный анализ позволяют не только прогнозировать вероятность дефолта заемщика, но и выявлять ключевые факторы, влияющие на кредитное поведение клиентов. Результаты исследования могут служить основой для дальнейших научных и прикладных работ в области разработки видеоигр и управления рисками.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация Streamlit [Электронный ресурс] // streamlit.io URL: <https://streamlit.io/> (дата обращения: 01.05.2025);
2. «Python Data Science Handbook» Джейк Вандер-Плас [Электронный ресурс] // jakevdp.github.io. URL: <https://jakevdp.github.io/PythonDataScienceHandbook/> (дата обращения: 02.05.2025);
3. Документация по Python [Электронный ресурс] // Python. URL: <https://docs.python.org/3/index.html/> (дата обращения: 01.05.2025);
4. Методические указания НИРС по дисциплине «Технологии машинного обучения».